System Design Document - Zentry

1. Introdução

1.1 Objetivo do documento

Esse System Design Document foi desenvolvido para descrever a arquitetura e as principais decisões técnicas do ZENTRY. Queremos apresentar a maneira em que o sistema será organizado de maneira que atenda aos requisitos funcionais, não funcionais e às regras de negócio previamente levantados. O documento cobre a topologia geral da solução, estratégia de escalabilidade para o volume de carga esperado, integrações com serviços externos, medidas de segurança e confiabilidade, além do processo de deploy.

Servirá como referência comum para a equipe de desenvolvimento durante a construção do MVP e documenta o motivo de cada escolha tecnológica. Dessa maneira, as decisões poderão ser revisadas conforme o produto evoluir.

1.2 Contextualização

Bares e restaurantes de pequeno e médio porte frequentemente utilizam ferramentas diferentes para controlar estoque, vendas, caixa, folha de pagamento e pedidos online. Essa fragmentação traz divergência entre o estoque físico e o registrado, dificuldade no fechamento de caixa, retrabalho no lançamento de informações, não integração entre vendas presenciais e online, também os relatórios financeiros pouco confiáveis. Por resultado, temos decisões de gestão importantes que acabam sendo baseadas em informações incompletas ou desatualizadas.

O ZENTRY é voltado para desenvolver aplicações web de gestão para restaurantes, de maneira que responda a esse problema. Queremos unificar tudo em uma única plataforma, manter os processos operacionais e administrativos do estabelecimento com uma base de dados centralizada. O acesso às funcionalidades é controlado por perfil de usuário (Administrador, Gerente ou Grupos de Telas customizados). O objetivo é reduzir erros e retrabalho, facilitar o fechamento de caixa e oferecer relatórios consolidados que apoiem a tomada de decisão.

Queremos atender dois públicos: a equipe do restaurante (garçons, operadores de caixa, gerentes, administradores e entregadores), aqueles que usam a aplicação diariamente, inclusive em tablets e celulares; e o consumidor final, que acessa diretamente o módulo de delivery para consultar o cardápio, realizar pedidos, pagamentos e acompanhar a entrega. Por isso, sem sombra de dúvidas, o projeto considera a LGPD, devido ao tratamento de dados pessoais, e o Código de Defesa do Consumidor, com a exigência de transparência de preços, taxas e promoções, registros confiáveis das operações e tratamento adequado de cancelamentos.

1.3 Escopo da arquitetura e funcionalidades contempladas

A arquitetura será um **monólito modular**, o frontend programando em **React,** o backend em **Java (Spring Boot)** exposto como API REST stateless e persistência em **MySQL**. Esta versão cobre o MVP e aponta os caminhos de evolução previstos, como múltiplas instâncias da API, réplicas de leitura e a separação de módulos em serviços próprios.

As funcionalidades contempladas estão organizadas nos módulos de **Cardápio, Estoque, Usuários e Permissões, Salão, Caixa, Financeiro, Delivery e Formas de Pagamento**.

O documento também trata da integração com o Mercado Pago, comunicação em tempo real por WebSocket, processamento assíncrono de tarefas internas e do dimensionamento para 1 milhão de escritas e 2 milhões de leituras, que usaremos nos testes de cargas.

Ficam fora do escopo a integração com plataformas de delivery de terceiros, a gestão de recursos humanos e o pagamento de folha salarial. Os detalhes de cada módulo estão nos documentos de requisitos e na especificação dos casos de uso.

2. Topologia e Arquitetura Geral - Monolítica Modular

2.1. Visão geral da arquitetura

Arquitetura escolhida

O ZENTRY usa a arquitetura de monólito modular. É uma única aplicação de backend, publicada de uma vez só, mas dividida por dentro em módulos independentes. Cada módulo representa uma área de negócio do sistema: Salão, Caixa, Cardápio, Estoque, Financeiro, Delivery, Formas de Pagamento, entre outros. Cada um tem suas regras, entidades e acesso a dados, e conversa com os outros por interfaces internas bem definidas.

A solução tem os seguintes componentes:

- **Frontend em React:** aplicação web servida como arquivos estáticos;
- **Backend em Java com Spring Boot:** expõe a API REST stateless e o canal WebSocket para as atualizações em tempo real;
- **MySQL:** fonte oficial dos dados do sistema;
- **Redis:** cache das leituras mais frequentes;
- **Nginx:** serve o frontend, faz o proxy reverso da API e cuida do HTTPS;
- **Mercado Pago:** provedor de pagamentos, acessado só pelo backend.

No MVP, todos esses componentes rodam em containers Docker em um único servidor (seção 5).

Justificativa da escolha

O grupo comparou três opções: monólito tradicional, microsserviços e monólito modular.

O monólito tradicional é o mais simples de construir. O problema é que, com 23 módulos, as regras de áreas diferentes acabariam misturadas no código. Isso deixa a manutenção mais difícil e atrapalha a divisão do trabalho entre os integrantes.

Os microsserviços dão isolamento e escala separada para cada serviço. Em troca, exigem vários deploys, comunicação pela rede entre os serviços, bancos separados e transações distribuídas. Esse custo não se justifica para o volume estimado (pico de 300 requisições por segundo, seção 3.1), para o tamanho da equipe e para o orçamento do projeto. Uma operação como fechar uma comanda, que mexe em comanda, pagamento e financeiro ao mesmo tempo, ficaria bem mais difícil de manter consistente.

O monólito modular junta o que serve das duas opções para o ZENTRY:

- um deploy só e um banco só, o que simplifica a publicação, o monitoramento e o uso de transações;
- módulos bem separados, o que facilita a manutenção e permite que cada integrante trabalhe na sua área sem interferir nas outras;
- espaço para evoluir: os módulos mais carregados, como o Delivery, podem virar serviços próprios no futuro sem reescrever o sistema (seção 3.3).

2.2. Frontend - React

Tecnologias escolhidas

O frontend é feito em React com TypeScript. O build é feito com o Vite, que roda em cima do Node.js e gera arquivos estáticos (HTML, CSS e JavaScript), servidos pelo Nginx. Por serem estáticos, esses arquivos podem ir para uma CDN em versões futuras (seção 3.3).

- **React Router:** navegação entre as telas, com rotas protegidas por perfil e carregamento de cada módulo só quando o usuário acessa;
- **TanStack Query:** busca e cache dos dados da API REST, com paginação, tratamento de erro e atualização da tela quando chega um evento pelo WebSocket ou depois de uma reconexão;
- **Zustand:** estado local da aplicação, como usuário logado, perfil e estabelecimento selecionado;
- **Ant Design:** biblioteca de componentes visuais voltada para sistemas administrativos, com tabelas, formulários, seletores de data e suporte a pt-BR;
- **@stomp/stompjs:** cliente WebSocket com STOMP, para receber as atualizações em tempo real do backend.

Organização da aplicação web

A aplicação é dividida pelos mesmos módulos de negócio do backend. Cada módulo (Salão, Caixa, Delivery, Estoque, Financeiro etc.) tem suas telas, componentes e chamadas à API. O que é usado em vários lugares, como tabela paginada, formulário e layout, fica numa área compartilhada. Os serviços de comunicação com a API e com o WebSocket também ficam nessa área.

As telas que aparecem dependem do perfil do usuário (Administrador, Gerente ou Grupo de Telas). Esconder telas no frontend é só para a experiência de uso. Quem autoriza de verdade é o backend, a cada requisição (seção 6.1).

Comunicação com o backend

- **API REST:** toda a comunicação é por HTTP com JSON, sempre por HTTPS. Depois do login, o token JWT vai em cada requisição. As listagens são paginadas, com no

máximo 50 registros por página. As chamadas passam pelo TanStack Query, que guarda o resultado em cache e evita buscar o mesmo dado várias vezes.

- **WebSocket com STOMP:** usado nas telas que precisam atualizar em tempo real, como o mapa do salão, o painel de pedidos de delivery, o acompanhamento do pedido pelo cliente e o status dos pagamentos (seção 4.2). O frontend se inscreve nos destinos de cada tela pelo @stomp/stompjs. Quando chega um evento, a consulta correspondente é invalidada e a tela busca o dado atualizado na API.
- **Queda do WebSocket:** o frontend tenta reconectar e busca o estado atual pela API REST. As operações essenciais continuam funcionando mesmo sem o WebSocket.
- **Pagamentos:** o React fala só com a API do ZENTRY. Ele nunca acessa as credenciais nem a API do Mercado Pago (seção 4.1.3).

2.3. Backend - Java

Tecnologias escolhidas

- **Java com Spring Boot:** base da aplicação;
- **Spring Web:** API REST;
- **Spring Security com JWT:** autenticação e autorização por perfil;
- **Spring Data JPA:** acesso ao MySQL, incluindo o controle de concorrência com @Version e o envio das leituras para a réplica com @Transactional(readOnly = true);
- **Spring Cache com Redis:** cache do cardápio e das configurações;
- **Spring WebSocket com STOMP:** notificações em tempo real;
- **@Scheduled:** processamento da tabela de eventos pendentes;
- **Spring Boot Actuator:** métricas para o Prometheus;
- **Flyway:** versionamento do banco **[a confirmar]**.

Definição da API REST stateless

A API não guarda sessão na memória do servidor. Cada requisição traz o token JWT, que dura pouco tempo e identifica o usuário e o perfil dele. O backend confere a permissão em toda chamada. Assim, qualquer instância da API atende qualquer requisição, e no futuro dá para colocar mais instâncias atrás de um balanceador de carga sem mudar o código (seção 3.2).

Regras da API:

- rotas organizadas por módulo (comandas, mesas, pedidos de delivery, insumos etc.);
- métodos HTTP de acordo com a operação: GET para consultar, POST para criar, PUT/PATCH para alterar e DELETE para remover;
- respostas em JSON, com códigos de status HTTP padronizados e um formato único para mensagens de erro;
- paginação obrigatória nas listagens, com no máximo 50 registros por página;
- toda comunicação por HTTPS, terminado no Nginx.

Organização dos módulos e camadas

Cada módulo de negócio é um pacote independente dentro da aplicação, e todos seguem as mesmas camadas:

- **Controller (API):** recebe a requisição, valida os dados de entrada e devolve a resposta;
- **Service (negócio):** aplica as regras de negócio e controla as transações;
- **Repository (persistência):** acessa o MySQL pelo JPA;
- **Domínio:** entidades e objetos que representam os dados do módulo.

Um módulo não acessa o repositório de outro módulo. Quando precisa de algo, usa a interface pública (service) do outro. Para ações que acontecem depois de uma operação concluída, como a baixa de insumos depois de uma venda, o sistema usa eventos internos e a tabela de eventos pendentes (seções 3.2 e 4.3.2).

O que é comum a todos os módulos fica numa camada compartilhada: segurança, tratamento de erros, configuração do cache, configuração do WebSocket e auditoria.

Responsabilidades dos componentes

Componente	**Responsabilidade**
Módulos de negócio	Regras de cada área (Salão, Caixa, Cardápio, Estoque, Financeiro, Delivery etc.)
Módulo de pagamentos	Única parte do sistema que fala com o Mercado Pago: cria as cobranças, recebe e valida os webhooks, controla a idempotência e os estados do pagamento (seção 4.1)
Segurança	Login, emissão e validação do JWT, conferência do perfil em cada requisição e bloqueio depois de várias tentativas de login erradas
Notificações em tempo real	Envia os eventos por WebSocket/STOMP para os clientes autorizados, só depois que a alteração foi gravada no MySQL
Processamento em segundo plano	Lê a tabela de eventos pendentes com @Scheduled, executa as tarefas e tenta de novo se falhar
Cache	Consulta o Redis antes do MySQL e remove a chave quando o dado muda (@CacheEvict)
Monitoramento	Expõe as métricas pelo Actuator para o Prometheus e o Grafana

2.4. Banco de dados e persistência - MySQL

Banco de dados escolhido e justificativa

O banco escolhido é o MySQL, com o motor InnoDB. Os motivos:

- **é relacional:** os dados do ZENTRY dependem muito uns dos outros (comanda, itens, pagamentos, mesas, insumos, lotes, lançamentos financeiros), e a consistência entre eles é obrigatória;

- **tem transações ACID:** uma operação que mexe em várias tabelas, como fechar uma comanda, grava tudo ou não grava nada (seção 3.2);
- **já cobre o que a escalabilidade pede:** replicação nativa para a réplica de leitura, particionamento por intervalo para as tabelas que não param de crescer e slow query log para o monitoramento;
- **funciona bem com o Spring Data JPA** e tem muita documentação, o que diminui o risco técnico para a equipe;
- **custo:** é gratuito e roda em container no mesmo servidor do MVP. Com o crescimento, pode ir para um serviço gerenciado (seção 5.1).

Estratégia de armazenamento e integridade dos dados

- **Fonte oficial:** o MySQL é a única fonte oficial dos dados. O Redis guarda só cópias temporárias do cardápio e das configurações. O WebSocket só avisa sobre mudanças que já estão gravadas no banco.
- **Integridade referencial:** chaves primárias, chaves estrangeiras, campos obrigatórios e restrições de unicidade garantem a consistência no próprio banco, sem depender só da aplicação.
- **Transações curtas:** operações em várias tabelas rodam numa única transação. A chamada ao Mercado Pago fica fora dela, para o banco não ficar com registros travados.
- **Duplicidade:** o índice único no identificador da transação do provedor impede que a mesma confirmação de pagamento seja gravada duas vezes.
- **Concorrência:** mesa, lote e comanda usam bloqueio otimista com campo de versão (@Version). Se dois usuários alterarem o mesmo registro, a segunda operação é recusada em vez de gravar um dado inconsistente.
- **Índices:** criados a partir do DER, para as consultas mais frequentes de cada módulo (seção 3.2).
- **Particionamento:** movimentacao_estoque, movimentacao_financeira e registro_auditoria são particionadas por mês.
- **Réplica de leitura:** consultas pesadas, como históricos e relatórios, vão para a réplica. Telas que precisam do dado que acabou de ser gravado leem do servidor principal.
- **Versionamento do esquema:** as alterações no banco são versionadas e aplicadas no deploy, sempre com backup antes (Flyway **[a confirmar]**).
- **Backup:** backup completo por dia e binlog, com as cópias fora do servidor principal (seção 6.2).
- **Dados sensíveis:** senhas guardadas só em hash e prazo de guarda dos dados pessoais conforme a LGPD (seção 6.1).

Relacionamento com os modelos de dados já elaborados

O esquema do banco vem direto do DER feito nas etapas anteriores. Cada entidade do DER vira uma tabela no MySQL e uma entidade JPA no módulo correspondente do backend:

- **Salão:** mesa, reserva;
- **Caixa:** comanda, item_comanda;
- **Formas de Pagamento:** pagamento;
- **Estoque:** insumo, lote, movimentacao_estoque;

- **Financeiro:** lancamento_financeiro, movimentacao_financeira;
- **Delivery:** pedido_delivery, cliente.

Cada módulo cuida das suas próprias tabelas, e o banco fica com a mesma separação do código. Fora o domínio de negócio, existem duas tabelas de apoio à arquitetura: a tabela de eventos pendentes (processamento em segundo plano) e o registro de auditoria.

3. Estratégia de Escalabilidade e Volumetria

3.1. Cenário de carga

O requisito estabelece uma proporção de duas leituras para cada escrita: 1 milhão de operações de escrita para 2 milhões de operações de leitura. Para dimensionar a solução, a equipe adota como premissa que esse volume, 3 milhões de requisições, ocorre ao longo de um dia de operação.

Como um restaurante não recebe requisições de forma uniforme, a estimativa considera a concentração do movimento nos horários de almoço (11h às 14h) e de jantar (19h às 22h):

- **Média do dia:** 3.000.000 ÷ 86.400 segundos ≈ 35 requisições por segundo.
- **Horário de movimento:** considerando que 70% das requisições ocorrem nessas 6 horas, 2.100.000 ÷ 21.600 segundos ≈ 100 requisições por segundo.
- **Pico:** aplicando um fator de 3 sobre o horário de movimento (sexta-feira à noite, promoções), chega-se a cerca de 300 requisições por segundo, das quais aproximadamente 200 são leituras e 100 são escritas.
- **Meta de projeto:** suportar 500 requisições por segundo, uma folga de cerca de 1,6 vez sobre o pico estimado, com tempo de resposta abaixo de 300 ms nas leituras e de 500 ms nas escritas, e taxa de erro inferior a 1%.

As leituras incluem a consulta ao cardápio, o status das mesas, o acompanhamento de pedidos de delivery, históricos e relatórios. As escritas incluem a abertura de comandas e o lançamento de itens, pagamentos e confirmações do provedor, movimentações de estoque, lançamentos financeiros e reservas.

Essas premissas serão validadas por testes de carga com a ferramenta k6, reproduzindo a proporção de duas leituras para cada escrita e a meta de 500 requisições por segundo.

3.2. Estratégias de escalabilidade

As estratégias abaixo estão organizadas em três frentes: a aplicação, as leituras, que representam dois terços do tráfego, e as escritas.

Aplicação

API stateless e escala horizontal: a autenticação é feita por token JWT, sem sessão guardada na memória do servidor. No MVP, o backend Spring Boot roda em uma única

instância, suficiente para o pico estimado de 300 requisições por segundo. Como a API não guarda estado, em versões posteriores será possível executar várias instâncias atrás de um balanceador de carga, adicionando novas conforme o uso de CPU, sem alterar o código. Nesse cenário, as notificações via WebSocket passarão a ser distribuídas entre as instâncias pelo Redis (publicação e assinatura), para que cada cliente receba os eventos independentemente da instância à qual está conectado.

Pool de conexões: o backend usa o balanceador com um limite de conexões (por exemplo, 20). Quando houver várias instâncias, esse limite por instância evita que o número total de conexões ultrapasse o máximo configurado no MySQL.

Leituras

Cache com Redis: os dados muito consultados e pouco alterados ficam em memória no Redis: o cardápio ativo (pratos, categorias, complementos e promoções vigentes) e as configurações do sistema. A aplicação consulta primeiro o Redis e, se o dado não estiver lá, busca no MySQL e o armazena (padrão cache-aside). Cada chave tem validade de 10 minutos e, quando um prato ou promoção é alterado, a chave correspondente é removida (anotação @CacheEvict do Spring), para que a próxima consulta já traga o dado atualizado. Status de mesas e de pedidos não vão para o cache, pois mudam a todo momento; essas atualizações chegam ao frontend por WebSocket (seção 4.2).

Réplica de leitura do MySQL: um servidor principal recebe todas as escritas e uma réplica recebe as consultas mais pesadas, como o histórico de pedidos (UC-DEL-008), o fluxo de caixa (UC-FIN-005) e os relatórios financeiros (UC-FIN-007). No Spring, os métodos marcados com @Transactional(readOnly = true) são direcionados à réplica. Como a réplica pode ter um atraso de alguns milissegundos, telas que precisam do dado recém-gravado, como a confirmação de um pagamento, continuam lendo do servidor principal.

Paginação obrigatória: nenhum endpoint devolve uma tabela inteira; as listagens retornam no máximo 50 registros por página.

Índices definidos a partir do DER: os índices são criados para as consultas mais frequentes de cada módulo:

- **comanda (status, aberta_em):** comandas abertas no caixa.
- **item_comanda (comanda_id):** itens de uma comanda.
- **lote (insumo_id, validade):** escolha do lote pela regra FEFO na retirada de insumos.
- **movimentacao_estoque (insumo_id, data_hora):** histórico de movimentações de um insumo.
- **pagamento (comanda_id) e índice único no identificador da transação do provedor:** pagamentos de uma comanda; o índice único também impede que uma confirmação duplicada do provedor seja registrada duas vezes.
- **lancamento_financeiro (situacao, vencimento):** contas em aberto e próximas do vencimento.
- **pedido_delivery (cliente_id):** histórico de pedidos do cliente.
- **reserva (data_hora):** reservas do dia no salão.

Escritas

Transações curtas: operações que alteram várias tabelas são gravadas em uma única transação. No fechamento de uma comanda, por exemplo, a atualização da comanda, o registro do pagamento e a movimentação financeira são concluídos juntos ou não são gravados. A chamada ao provedor de pagamento fica fora da transação, para que o banco não mantenha registros bloqueados enquanto aguarda uma resposta externa.

Processamento em segundo plano (tabela de eventos): tarefas que não precisam acontecer no mesmo instante, como a baixa automática de insumos por venda (UC-CAR-006), o lançamento financeiro das vendas (UC-FIN-011) e a geração de alertas, não são executadas durante a requisição do usuário. Na mesma transação do fechamento da comanda, o sistema grava um registro em uma tabela de eventos pendentes; um processo agendado no próprio backend (@Scheduled) lê esses registros, executa as tarefas e tenta novamente em caso de falha. Assim, a resposta ao operador de caixa é rápida e nenhuma baixa é perdida, sem exigir infraestrutura adicional (padrão conhecido como outbox).

Controle de concorrência: as entidades disputadas por vários usuários ao mesmo tempo (mesa, lote e comanda) usam bloqueio otimista, por meio de um campo de versão (@Version do JPA). Se dois garçons tentarem ocupar a mesma mesa, ou duas retiradas consumirem o mesmo lote, a segunda operação é recusada e refeita com o dado atualizado, em vez de gravar uma informação inconsistente.

Tabelas de crescimento contínuo: as tabelas movimentacao_estoque, movimentacao_financeira e registro_auditoria são particionadas por mês (particionamento por intervalo do MySQL). Consultas por período leem apenas as partições necessárias, e os meses antigos podem ser arquivados sem afetar a operação.

Monitoramento

O MySQL registra as consultas com mais de 200 ms (slow query log), e a aplicação expõe métricas pelo Spring Boot Actuator, coletadas pelo Prometheus e exibidas no Grafana: tempo de resposta, taxa de acerto do cache, uso do pool de conexões e tamanho da fila de eventos pendentes. Os ajustes de índices, cache e número de instâncias são orientados por essas métricas e pelos testes de carga.

3.3. Justificativa técnica

Como as estratégias atendem ao cenário: o pico estimado é de 300 requisições por segundo e a meta de projeto é de 500. As leituras, que representam dois terços desse tráfego, são absorvidas em grande parte pelo cache, no caso do cardápio, e pela réplica, no caso de históricos e relatórios. Com isso, o servidor principal do MySQL recebe essencialmente as escritas, cerca de 100 por segundo no pico, volume que um MySQL com índices adequados atende com folga, o que será confirmado nos testes de carga. As escritas ficam leves porque a requisição do usuário grava apenas o essencial e deixa o restante para o processamento em segundo plano. No MVP, uma única instância da API

atende esse volume; como a API não guarda estado, a capacidade de atendimento poderá crescer em versões posteriores adicionando instâncias.

Limitações: o servidor principal do MySQL é o limite para as escritas e um ponto único de falha, mitigado pelos backups e pela recuperação descritos na seção 6.2. A réplica pode apresentar um pequeno atraso em relação ao principal. Se a remoção de uma chave do cache falhar, o cardápio pode ficar desatualizado por até 10 minutos, limite dado pela validade da chave. Por fim, a premissa de distribuição diária do volume precisa ser confirmada; se o volume ocorrer em um intervalo menor, a meta de projeto deve ser recalculada.

Evolução: com o aumento da demanda, a API pode passar a rodar em várias instâncias com balanceador de carga, a tabela de eventos pode ser substituída por um serviço de mensageria, novas réplicas de leitura podem ser adicionadas e os módulos mais carregados, como o Delivery, podem ser separados em serviços próprios, o que a organização em monólito modular já facilita, já para o frontend em React, por ser composto de arquivos estáticos, pode ser distribuído por uma CDN.

4. Integrações e Comunicação Síncrona/Assíncrona

4.1. Integração com API de pagamentos

4.1.1. Provedor escolhido

Propõe-se a utilização do Mercado Pago como provedor de pagamentos da ZENTRY, considerando sua atuação no mercado brasileiro, a disponibilidade de APIs REST e o suporte a métodos como Pix e cartão de crédito.

A integração permitirá delegar o processamento financeiro a um serviço especializado, enquanto a ZENTRY será responsável pelo gerenciamento dos pedidos e pelo controle dos estados dos pagamentos. É válido acrescentar que a escolha definitiva deverá considerar os requisitos técnicos, as tarifas e as condições de utilização do provedor.

4.1.2. Fluxo de criação e confirmação do pagamento

O usuário selecionará o método de pagamento na interface React, que enviará uma requisição à API da ZENTRY. O backend Java com Spring Boot validará o pedido, calculará o valor e registrará a tentativa de pagamento no MySQL antes de solicitar a cobrança ao provedor.

Após a criação, o sistema armazenará o identificador da transação e apresentará as informações necessárias ao usuário. A confirmação ocorrerá por meio de webhooks enviados pelo provedor ao backend, que validará a notificação e atualizará o estado do pagamento no banco de dados. Por fim, o frontend consultará a API do ZENTRY para apresentar o resultado atualizado.

O fluxo contemplará estados como pendente, aprovado, recusado, cancelado e expirado, garantindo que somente pagamentos devidamente confirmados sejam considerados concluídos.

4.1.3. Comunicação entre ZENTRY e provedor

A comunicação será realizada por meio de APIs REST sobre HTTPS, com as requisições externas concentradas no backend Java. O React interagirá exclusivamente com a API interna da ZENTRY, sem acesso às credenciais secretas do provedor.

Na arquitetura monolítica modular, a integração ficará isolada em um módulo de pagamentos, responsável pela comunicação com o serviço externo e pelo gerenciamento das transações. Os dados necessários serão persistidos no MySQL, e as credenciais serão armazenadas em configurações protegidas, externas ao código-fonte.

4.1.4. Tratamento de falhas, duplicidades e segurança

O sistema utilizará mecanismos de idempotência para evitar cobranças duplicadas em solicitações repetidas e controles de persistência para impedir o processamento duplicado de notificações. Em caso de timeout ou indisponibilidade do provedor, o pagamento permanecerá em verificação até que seu resultado possa ser confirmado, evitando novas cobranças indevidas.

A segurança será garantida por meio de HTTPS, proteção das credenciais no backend, validação da autenticidade dos webhooks e conferência dos valores e identificadores das transações. Também serão previstos limites de tempo, retentativas controladas e registros de auditoria, sem exposição de dados sensíveis.

4.1.5. Ambiente de homologação

Antes da entrada em produção, a integração será validada no ambiente de testes do provedor, utilizando credenciais e dados de pagamento específicos para simulação, sem movimentação financeira real.

Serão testados pagamentos aprovados, recusados e pendentes, além de falhas de comunicação, notificações duplicadas ou inválidas e tentativas simultâneas. A homologação deverá verificar a consistência dos registros no MySQL, a atualização correta dos estados e a segurança do processamento. Após a validação, serão configuradas as credenciais e os endpoints de produção, respeitando os requisitos do provedor.

4.2. Comunicação via WebSocket

4.2.1. Casos de uso que demandam atualizações em tempo real

A comunicação via WebSocket será utilizada nas funcionalidades da ZENTRY que necessitam de atualização rápida entre diferentes interfaces do sistema, evitando consultas periódicas desnecessárias à API REST.

Os principais casos de uso contemplados são:

- **UC-DEL-002 — Realizar Pedido de Delivery:** notificação do painel interno quando um novo pedido for registrado.
- **UC-DEL-003 — Gerenciar Pedidos de Delivery:** comunicação das alterações de status realizadas durante o atendimento.
- **UC-DEL-005 — Realizar Entrega do Pedido:** notificação sobre o início e a conclusão da entrega.
- **UC-DEL-006 — Acompanhar Pedido:** atualização do status do pedido e das informações de entrega para o cliente.
- **UC-SAL-006 — Consultar Status das Mesas:** atualização do mapa do salão conforme mudanças no estado das mesas.
- **UC-CXA-003 — Fechar Comanda e Processar Pagamento:** atualização do status da comanda e da disponibilidade da mesa após a conclusão do pagamento.
- **UC-SAL-009 — Confirmar Chegada do Cliente (Check-in):** atualização da ocupação da mesa após a chegada do cliente.
- **UC-SAL-010 — Tratar Reserva Não Comparecida (No-show):** notificação da liberação da mesa após o cancelamento automático da reserva.

Adicionalmente, o mecanismo poderá comunicar alterações de pagamento nos casos **UC-PAG-002 — Processar Pagamento de Venda** e **UC-PAG-004 — Cancelar ou Estornar Pagamento**, mantendo a confirmação e a persistência do estado sob responsabilidade do backend.

Outros casos de uso, como **UC-DEL-004 — Gerenciar Entregadores**, **UC-SAL-007 — Realizar Reserva de Mesa**, **UC-SAL-008 — Editar ou Cancelar Reserva**, **UC-EST-001 — Registrar Movimentação de Estoque (Entrada/Retirada)**, **UC-EST-EQP-002 — Registrar Ocorrência/Falha de Equipamento** e **UC-FIN-010 — Consultar Alertas Financeiros**, poderão utilizar o mesmo mecanismo quando houver necessidade de notificar outras interfaces sobre alterações relevantes.

4.2.2. Justificativa técnica

O WebSocket será utilizado como canal complementar à API REST, permitindo que o backend envie eventos aos clientes conectados após o processamento de alterações relevantes. Essa abordagem é adequada aos casos de uso que exigem acompanhamento contínuo de pedidos, entregas, mesas e pagamentos.

A utilização de WebSocket com STOMP permitirá organizar as mensagens por destinos e inscrições, enquanto um broker compartilhado será previsto para distribuir os eventos entre múltiplas instâncias do backend.

4.2.3. Eventos e componentes envolvidos

Os módulos de Delivery, Caixa, Salão, Formas de Pagamento e demais módulos envolvidos nas funcionalidades selecionadas serão responsáveis por gerar eventos após as alterações pertinentes serem processadas e persistidas.

O backend Java/Spring Boot validará as operações, registrará o estado oficial no MySQL e encaminhará as notificações aos clientes autorizados. O frontend React receberá os eventos e atualizará as interfaces correspondentes.

Os eventos previstos incluem a criação e atualização de pedidos, mudanças no estado das entregas, alterações na disponibilidade das mesas e mudanças no status dos pagamentos.

4.2.4. Gerenciamento de conexões e tratamento de desconexões

As conexões WebSocket serão autenticadas e submetidas a controles de autorização, restringindo o recebimento de mensagens conforme o perfil do usuário, o estabelecimento e os recursos aos quais possui acesso.

Para suportar múltiplas instâncias do backend, será previsto um broker compartilhado para distribuição dos eventos. Em caso de desconexão, o frontend tentará restabelecer a conexão e consultará a API REST para recuperar o estado atual, utilizando os dados persistidos no MySQL como fonte oficial.

Serão considerados mecanismos de detecção de conexões inativas, tratamento de mensagens duplicadas e monitoramento de falhas. A indisponibilidade temporária do WebSocket não deverá impedir as operações essenciais realizadas pela API REST.

4.3. Outras comunicações

4.3.1. Processamento assíncrono

O sistema poderá utilizar processamento assíncrono para tarefas que não precisem ser concluídas durante a requisição do usuário, reduzindo o tempo de resposta da API REST e permitindo melhor gerenciamento de operações demoradas. A adoção desse mecanismo será definida conforme os requisitos de cada funcionalidade.

4.3.2. Eventos e comunicação interna

A comunicação entre os módulos do monólito modular ocorrerá preferencialmente por meio de interfaces internas bem definidas. Quando necessário, poderão ser utilizados eventos internos para desacoplar ações decorrentes de operações concluídas, como uma venda ou alteração de pedido. A adoção de mecanismos adicionais de mensageria dependerá dos requisitos de escalabilidade, confiabilidade e processamento.

5. Estratégia de Deploy e CI/CD

5.1. Infraestrutura de hospedagem

No MVP, o ZENTRY será hospedado em um servidor virtual (VPS) em nuvem, em um único servidor, o que é suficiente para o pico estimado de 300 requisições por segundo com uma única instância da API (seção 3.2). O servidor executa o backend em Spring Boot, o

MySQL, o Redis e o Nginx. O Nginx serve os arquivos estáticos do React, atua como proxy reverso da API e cuida do HTTPS. O Prometheus e o Grafana ficam no mesmo servidor, para acompanhar as métricas. O provedor da VPS ainda precisa ser escolhido pelo grupo [a confirmar]. Com o crescimento do uso, o MySQL pode migrar para um serviço gerenciado com réplica, e o frontend pode passar para uma CDN, como já previsto na seção 3.3.

5.2. Containerização e configuração

Cada componente roda em um container Docker, orquestrado por Docker Compose no MVP. A mesma imagem é usada em homologação e produção, o que evita o problema de "funciona na minha máquina". Existem três ambientes: desenvolvimento (na máquina de cada integrante), homologação e produção. As configurações de cada um ficam em variáveis de ambiente, com perfis do Spring. Credenciais como senha do banco, chave de assinatura do JWT e chaves do Mercado Pago ficam nos secrets do GitHub e em um arquivo de ambiente no servidor, nunca no código. A homologação usa só as credenciais de teste do Mercado Pago (seção 4.1.5).

5.3. Integração e entrega contínuas

O código fica no repositório oficial do GitHub. A branch `main` é protegida: toda alteração entra por Pull Request, com revisão de pelo menos um colega.

A integração contínua roda no GitHub Actions a cada Pull Request: compila o backend, roda os testes automatizados (unitários e de integração), compila o frontend e roda os testes do React. O merge só é liberado se tudo passar.

Depois do merge na `main`, o pipeline gera as imagens Docker e publica automaticamente em homologação. Lá o grupo valida a versão e roda um teste de carga com o k6, como previsto na seção 3.1. A publicação em produção só acontece após aprovação manual do PO.

Na publicação, o container é substituído pela versão nova, com uma breve interrupção, feita fora do horário de movimento. Se a versão apresentar problema, o grupo volta para a imagem anterior, que fica guardada com sua tag. As alterações no banco são versionadas (com Flyway **[a confirmar]**) e aplicadas no deploy, sempre com backup antes.

O GitHub Actions foi escolhido porque o código já está no GitHub e dispensa um servidor de CI separado, o que combina com o orçamento enxuto do projeto.

6. Segurança e Confiabilidade

6.1. Segurança

Para entrar no ZENTRY, o usuário faz login com e-mail e senha e recebe um token JWT que vale por pouco tempo. As senhas ficam no banco só em forma de hash, então ninguém consegue ler a senha de verdade. Se alguém errar a senha várias vezes seguidas, o login é bloqueado por um tempo.

Cada pessoa só acessa o que o perfil dela permite (Administrador, Gerente ou Grupo de Telas). Essa conferência acontece no backend a cada requisição, e não só escondendo botões na tela, porque esconder no frontend não impede ninguém de chamar a API direto.

Na parte de LGPD, o sistema guarda só os dados que a operação realmente precisa e não vende nada para terceiros. Cada tipo de dado tem uma base legal e um prazo de guarda, e o titular pode pedir para apagar os dados dele. O ambiente de homologação usa apenas dados inventados, e toda a comunicação passa por HTTPS.

As chaves do Mercado Pago ficam só no backend e nunca vão para o React. Os webhooks têm a origem conferida antes de serem aceitos, e os pagamentos usam idempotência, como já está na seção 4.1.4. Os logs não registram dados sensíveis.

6.2. Confiabilidade

Quando uma operação mexe em várias tabelas, como fechar uma comanda, ela roda numa transação só: ou grava tudo ou não grava nada. A chamada ao Mercado Pago fica fora dessa transação, para o banco não ficar travado esperando resposta. Se o provedor demorar ou sair do ar, o pagamento fica em verificação até a confirmação chegar. As tarefas que rodam depois, como a baixa de insumos, ficam salvas na tabela de eventos e são tentadas de novo se falharem. Se o WebSocket cair, o frontend reconecta e busca o estado atual pela API.

Sobre backup, o MySQL terá um backup completo por dia e o registro das alterações (binlog), assim dá para recuperar os dados até um ponto bem perto do problema. As cópias ficam fora do servidor principal, e a restauração é testada de tempos em tempos. Antes de cada atualização do banco também é feito um backup. Como o MySQL principal é o ponto único de falha, apontado na seção 3.3, é essa rotina que protege os dados. A frequência exata e o tempo de guarda ainda precisam ser definidos pelo grupo.

Para o monitoramento, o Actuator, o Prometheus e o Grafana mostram o tempo de resposta, a taxa de erro, o uso das conexões e o tamanho da fila de eventos. Se a taxa de erro passar de 1%, que é a meta do projeto, o grupo é avisado. O slow query log do MySQL e o registro de auditoria ajudam a achar a causa dos problemas.

7 .Registro de contribuições

Cada integrante deve preencher a tabela abaixo com as atividades realizadas durante a elaboração deste documento, informando a seção trabalhada, uma breve descrição da atividade e a data e o horário em que ela foi realizada. Caso um integrante contribua em diferentes momentos ou seções, deverá adicionar uma nova linha para cada registro.

Integrante	Seção do documento	Descrição da atividade realizada	Data	Horário

João Marcos	3.1	Defini a tecnologia para escalabilidade (Redis)	26/10/26	21:37
João Marcos	3.0	Removi o tópico 3.2, inclui dentro do 3.1 para facilitar o entendimento	26/10/26	22:42
Gustavo Ichikawa	4.1	Elaboração da seção de integração com API de pagamentos, contemplando provedor, fluxo de pagamento, comunicação, segurança, tratamento de falhas e homologação.	26/10/26	21:37
Gustavo Ichikawa	4.2	Reestruturação da seção 4.2, definindo os casos de uso que necessitam de atualizações em tempo real via WebSocket, a integração com o backend e o broker compartilhado, além de medidas de segurança, gerenciamento de conexões e recuperação após desconexões.	27/10/26	19:41
Gustavo Ichikawa	4.3	Definição das diretrizes para processamento assíncrono e comunicação interna entre módulos, visando escalabilidade e desacoplamento quando necessário.	27/10/26	21:09
João Marcos	3.3	Justificativa das escolhas tecnológicas da parte de escalabilidade	27/10/20 26	21:15
Stephany Christine	1.1	Resumo do objetivo do deste documento	29/10/20 26	18:27
Stephany Christine	1.2	Contexto com o que é o Zentry e qual problema queremos resolver	29/10/20 26	18:50
Stephany Christine	1.3	Resumo do Escopo e Arquitetura do projeto	29/10/20 26	19:23

Gustavo Dena	2	Definição das justificativas das nossas arquiteturas e stacks	29/10/2026	22:42